

AVR-Based Smart Home Automation and Security System

Undergraduate Capstone Project Archive

Ali Moltajaei Farid

May 2009

Contents

1	Introduction	3
2	System Architecture	5
2.1	Transmitter Unit	5
2.2	Receiver Unit	6
2.3	Communication Protocol	7
2.4	Sensor and Monitoring Subsystem	7
2.4.1	Temperature Monitoring	7
2.4.2	Light Detection	7
2.4.3	Security Monitoring	7
2.5	User Interface	7
2.6	System Operation Overview	8
3	Software Design and Implementation	9
3.1	Software Structure	9
3.2	Microcontroller Initialization	10
3.3	Main Control Loop	10
3.4	Real-Time Clock Management	11
3.5	Sensor Acquisition	11
3.5.1	Temperature Sensor	11
3.5.2	Photocell Sensors	11
3.5.3	Laser Protection Sensor	11
3.6	Communication Software	12
3.7	Relay Scheduling Algorithm	12
3.8	User Interface Software	12
3.9	Password and Security System	13
3.10	Receiver Control Logic	13
3.11	Software Limitations	14
4	Hardware Implementation	15
4.1	System Hardware Overview	15
4.2	Transmitter Board Hardware	15
4.2.1	Microcontroller Unit	16
4.2.2	Real-Time Clock Module	17

4.2.3	LCD Interface	17
4.2.4	Keypad Interface	17
4.2.5	Temperature Sensor	18
4.2.6	Photocell Sensors	18
4.2.7	Laser Protection Sensor	18
4.3	Receiver Board Hardware	18
4.3.1	Relay Driver Circuit	19
4.3.2	Buzzer Alarm	19
4.4	Power Supply Design	19
4.5	Communication Interface	20
4.6	Hardware Design Considerations	20
5	Experimental Results and System Evaluation	21
5.1	Experimental Setup	21
5.2	Relay Scheduling Evaluation	22
5.3	Temperature Control Performance	22
5.4	Light Detection Evaluation	22
5.5	Security System Evaluation	23
5.6	Communication Reliability	23
5.7	User Interface Evaluation	24
5.8	System Stability	24
5.9	System Limitations	24
5.10	Possible Improvements	25
5.11	Summary	25

Chapter 1

Introduction

Home automation and embedded monitoring systems have become an essential part of modern intelligent infrastructure. The integration of sensing, control, communication, and security functionalities into low-cost embedded platforms has enabled the development of compact and efficient smart environments for residential and industrial applications. Although contemporary smart systems are commonly based on wireless networks and high-performance processors, earlier generations of embedded automation systems relied heavily on resource-constrained microcontrollers and custom communication protocols.

This project presents the design and implementation of an AVR-based smart home automation and security system developed as an undergraduate capstone project in 2009. The system was designed around Atmel AVR microcontrollers and implemented using the C programming language within the CodeVisionAVR development environment. The primary objective of the project was to create an integrated embedded platform capable of performing environmental monitoring, scheduled device control, security management, and user interaction through a standalone hardware architecture.

The system consists of two main subsystems: a transmitter/controller unit and a receiver/control unit communicating through asynchronous serial communication (USART). The transmitter board acts as the central controller responsible for sensor acquisition, real-time scheduling, password verification, user interface management, and automation logic. The receiver board interprets transmitted command packets and controls external devices such as relays, lighting systems, cooling systems, and alarm outputs.

Several hardware modules were integrated into the system, including a DS1307 real-time clock (RTC), analog temperature and light sensors, laser-based security detection, relay drivers, buzzer alarms, keypad inputs, and alphanumeric LCD displays. The system supports programmable scheduling for multiple relays, automatic temperature-based cooling control, environmental light detection, and a password-protected security mode. Additionally, the software includes a Gregorian-to-Shamsi (Persian) calendar conversion algorithm to provide localized date representation.

From a software engineering perspective, the project demonstrates the implementation of embedded system concepts such as peripheral interfacing, analog-to-digital conversion, serial communication protocols, menu-driven user interfaces, and real-time event handling within the constraints of limited hardware resources. Despite being developed using polling-based

execution and simple checksum validation mechanisms, the project successfully achieved reliable operation on low-power 8-bit microcontrollers.

This report documents the hardware architecture, software design, communication protocol, automation logic, and security features of the system. Furthermore, it provides a retrospective technical evaluation of the design choices, implementation limitations, and possible modern improvements that could be applied using current embedded systems technologies.

Chapter 2

System Architecture

The proposed smart home automation system was designed using a distributed embedded architecture consisting of two independent AVR-based processing units connected through asynchronous serial communication. The architecture separates the system into a high-level controller subsystem and a low-level actuator subsystem in order to simplify device management and improve modularity.

The overall system structure is composed of the following primary modules:

- Transmitter/Controller Unit
- Receiver/Actuator Unit
- Sensor Interfaces
- User Interface Modules
- Communication Interface
- Security and Alarm Subsystem

The transmitter unit operates as the central processing controller of the system. It continuously acquires data from environmental sensors, processes user inputs, manages scheduling operations, and generates command packets for remote device control. The receiver unit operates as an execution layer responsible for interpreting received commands and controlling output peripherals such as relays, lighting systems, cooling devices, and alarm outputs.

The two subsystems communicate through a USART serial interface configured at 9600 baud. A custom packet-based communication protocol was implemented in software to ensure synchronization between the controller and actuator units. Each transmitted frame contains command bytes associated with different automation events together with a checksum value for basic transmission validation.

2.1 Transmitter Unit

The transmitter board represents the intelligent core of the system. It integrates multiple software and hardware functionalities, including:

- Real-time clock management
- User menu handling
- Password verification
- Relay scheduling
- Sensor monitoring
- Automation decision making
- Communication packet generation

The controller continuously monitors environmental conditions using analog sensor inputs connected to the microcontroller ADC channels. Based on predefined thresholds and programmed schedules, control commands are transmitted to the receiver board.

The transmitter subsystem incorporates the following hardware components:

Table 2.1: Transmitter Hardware Components

Component	Function
ATmega AVR Microcontroller	Main system controller
DS1307 RTC	Time and date management
LCD Display	User interface and monitoring
Keypad Inputs	User configuration and navigation
Temperature Sensor	Cooling automation
Photocell Sensors	Light condition monitoring
Laser Sensor	Intrusion detection
USART Interface	Communication with receiver

2.2 Receiver Unit

The receiver subsystem is responsible for direct interaction with output devices. It receives command packets from the transmitter unit and activates corresponding hardware outputs.

The receiver performs the following tasks:

- Serial packet reception
- Checksum verification
- Relay activation/deactivation
- Lighting control
- Cooling system control
- Alarm and buzzer control
- LCD status indication

The receiver architecture was intentionally designed to remain simple and lightweight in order to minimize processing overhead and improve response reliability.

2.3 Communication Protocol

A custom serial communication protocol was implemented between the transmitter and receiver boards. Communication begins with a synchronization byte followed by multiple command bytes and a checksum field.

The communication frame structure is summarized in Table 2.1.

Table 2.2: Communication Packet Structure

Byte Position	Description
Byte 0	Start byte (0xFF)
Byte 1–12	Command data
Byte 13	Checksum

The checksum is generated by summing all transmitted command bytes and comparing the result at the receiver side. Although this mechanism provides only basic error detection capability, it was sufficient for short-range serial communication in the implemented prototype.

2.4 Sensor and Monitoring Subsystem

The system integrates several analog and digital sensors for environmental monitoring and security applications.

2.4.1 Temperature Monitoring

The temperature sensor is connected to an ADC channel of the transmitter microcontroller. The measured temperature is continuously displayed on the LCD interface and compared against a configurable threshold value to control cooling operations automatically.

2.4.2 Light Detection

Photocell sensors are used to determine indoor and outdoor illumination conditions. The system can automatically trigger lighting-related events depending on measured voltage levels.

2.4.3 Security Monitoring

A laser-based protection mechanism was implemented for intrusion detection. Interruption of the laser signal activates an alarm buzzer when the protection mode is enabled.

2.5 User Interface

A menu-driven user interface was implemented using an alphanumeric LCD and multiple keypad buttons. The interface allows the user to:

- Configure date and time
- Program relay schedules

- Modify temperature thresholds
- Enable or disable protection mode
- Change system passwords

The menu system was designed to operate entirely on embedded hardware without requiring external software or computer interaction.

2.6 System Operation Overview

During normal operation, the transmitter continuously performs the following sequence:

1. Read sensor values
2. Acquire current date and time
3. Compare conditions against scheduled events
4. Generate automation commands
5. Transmit command packets to receiver
6. Update LCD information
7. Monitor security conditions

The receiver processes incoming packets and immediately updates output states based on received commands.

This architecture enabled the implementation of a complete standalone smart automation platform using low-cost 8-bit embedded hardware.

“`latex id=`”5n6wd8”

Chapter 3

Software Design and Implementation

The software architecture of the proposed smart home automation system was implemented using the C programming language within the CodeVisionAVR integrated development environment. The system software was divided into two major modules corresponding to the transmitter and receiver subsystems. Each subsystem was designed to perform dedicated tasks while maintaining synchronized communication through a serial interface.

The implementation was developed for AVR microcontrollers operating at an 8 MHz clock frequency. The software directly accesses hardware peripherals including GPIO ports, ADC modules, USART communication registers, timers, and external peripheral interfaces such as I²C.

3.1 Software Structure

The software was organized into multiple functional components responsible for different embedded system operations. These components include:

- Hardware initialization
- Sensor acquisition
- USART communication
- LCD interface management
- Real-time scheduling
- Password management
- Security monitoring
- Automation control logic

The transmitter software continuously executes a polling-based control loop responsible for reading sensor data, updating system states, and transmitting control commands to the receiver unit.

The receiver software executes a simplified event-processing loop dedicated to packet reception and actuator control.

3.2 Microcontroller Initialization

At system startup, all hardware peripherals are initialized before entering the main execution loop. Initialization routines configure:

- Input and output ports
- USART communication parameters
- ADC conversion settings
- LCD interface
- I²C communication
- Real-time clock interface
- Timer modules

The USART communication interface was configured for asynchronous communication with the following parameters:

- Baud rate: 9600
- Data bits: 8
- Stop bits: 1
- Parity: None

The ADC subsystem was configured to use the AVCC voltage reference for analog sensor acquisition.

3.3 Main Control Loop

The transmitter operates using a continuous polling-based execution loop. During each iteration, the system performs the following operations:

1. Read current time and date from RTC
2. Acquire analog sensor values
3. Update LCD information
4. Compare scheduled events
5. Evaluate automation conditions
6. Generate communication packets
7. Transmit commands to receiver

8. Monitor security status
9. Process user inputs

This execution model simplified software implementation and reduced synchronization complexity for the limited hardware resources available at the time of development.

3.4 Real-Time Clock Management

The DS1307 Real-Time Clock module was interfaced using the I²C communication protocol. The RTC subsystem provides continuous date and time information required for scheduling and automation operations.

The software supports:

- Time display
- Date display
- User-adjustable clock settings
- Relay scheduling
- Persian calendar conversion

The Gregorian-to-Shamsi conversion algorithm was implemented manually using lookup tables and date transformation logic.

3.5 Sensor Acquisition

The system integrates multiple analog sensors connected to the AVR ADC channels.

3.5.1 Temperature Sensor

The temperature monitoring subsystem continuously samples analog temperature data and converts the measured value into a displayable format. The measured temperature is compared against a configurable threshold value in order to automatically control the cooling system.

3.5.2 Photocell Sensors

Photocell sensors were used to monitor environmental illumination conditions. The measured voltages determine whether the environment is classified as dark or bright.

These measurements are used for lighting-related automation decisions.

3.5.3 Laser Protection Sensor

A laser-based intrusion detection mechanism was implemented as part of the security subsystem. Interruption of the laser signal activates an alarm condition when protection mode is enabled.

3.6 Communication Software

Communication between the transmitter and receiver boards was implemented using USART serial communication.

The transmitter constructs command packets containing automation events and device control instructions. Each packet consists of:

- Synchronization byte
- Command data bytes
- Checksum field

The checksum value is calculated by summing all transmitted command bytes.

The receiver validates incoming packets before executing control actions. Invalid packets are ignored in order to reduce communication errors.

3.7 Relay Scheduling Algorithm

The automation subsystem supports programmable scheduling for multiple relays.

Each relay contains:

- Activation time
- Deactivation time

The software continuously compares the current RTC time against stored scheduling parameters. When a scheduling condition is satisfied, the corresponding command packet is transmitted to the receiver subsystem.

This mechanism allows autonomous device control without external supervision.

3.8 User Interface Software

A menu-driven user interface was implemented using an alphanumeric LCD display and keypad buttons.

The user interface supports:

- Date and time configuration
- Relay programming
- Temperature threshold adjustment
- Password modification
- Protection mode activation

Navigation is performed through keypad inputs connected directly to the AVR GPIO pins. The LCD continuously displays:

- Current time
- Current date
- Temperature readings
- Security warnings
- Device status information

3.9 Password and Security System

A password-based authentication mechanism was implemented to restrict access to sensitive system settings.

The password subsystem supports:

- Password verification
- Password modification
- Protection mode locking

When protection mode is enabled, intrusion events detected by the laser sensor activate the alarm subsystem until the correct password is entered.

3.10 Receiver Control Logic

The receiver subsystem continuously waits for incoming serial packets from the transmitter board.

After receiving a valid communication frame, the receiver performs:

- Checksum verification
- Device state decoding
- Relay switching
- Cooler control
- Lighting control
- Alarm activation

The receiver directly controls output pins connected to relays and external devices.

3.11 Software Limitations

Although the implemented software successfully achieved the required functionality, several limitations were identified:

- Polling-based execution instead of interrupt-driven processing
- Limited communication error detection capability
- Extensive use of global variables
- Lack of persistent EEPROM configuration storage
- Monolithic software structure

These limitations were acceptable within the scope and hardware constraints of the original project but could be improved significantly using modern embedded software design methodologies.

“`latex id=h7q2vx`”

Chapter 4

Hardware Implementation

The hardware implementation of the proposed smart home automation system was designed using low-cost AVR-based embedded components and standard peripheral modules. The system architecture consists of two separate hardware boards interconnected through USART serial communication: the transmitter/controller board and the receiver/actuator board.

The hardware design focused on modularity, simplicity, and reliable integration of sensing, control, monitoring, and security functionalities.

4.1 System Hardware Overview

The complete hardware platform integrates multiple embedded modules responsible for:

- Environmental sensing
- Time management
- User interaction
- Device control
- Security monitoring
- Serial communication

The transmitter board acts as the intelligent processing unit while the receiver board functions as the output control interface for external devices.

A simplified overview of the hardware architecture is shown in Figure 4.1.

4.2 Transmitter Board Hardware

The transmitter subsystem was implemented using an AVR microcontroller connected to multiple sensors, interface modules, and communication peripherals.

The transmitter hardware is responsible for:

- Sensor acquisition

Overall System Hardware Architecture

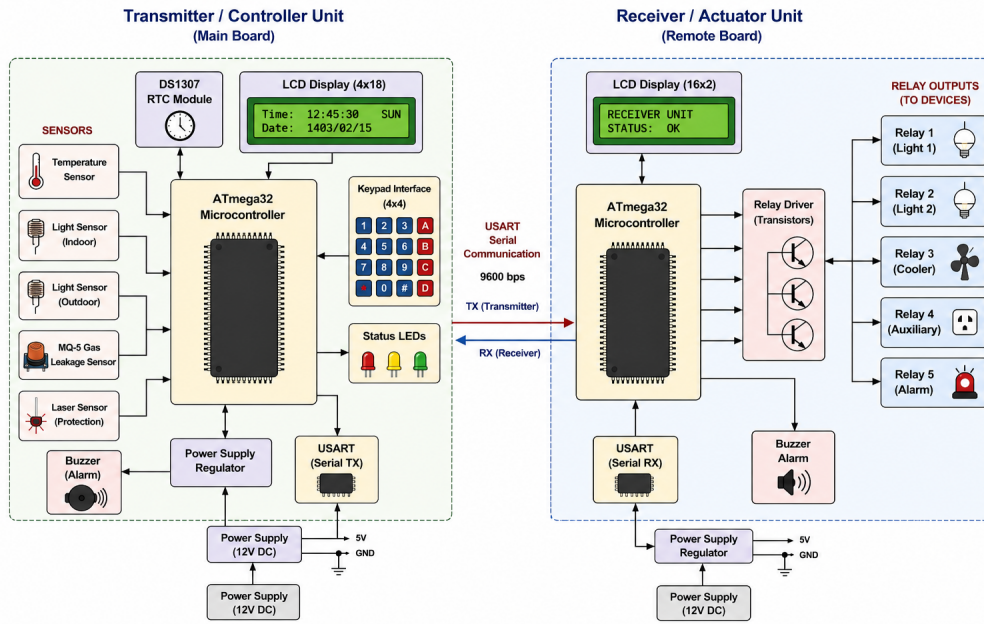


Figure 4.1: Overall System Hardware Architecture

- User interface management
- RTC communication
- Automation processing
- Command transmission

4.2.1 Microcontroller Unit

The main controller was implemented using an Atmel AVR microcontroller from the ATmega family. The microcontroller performs all central processing tasks including sensor monitoring, scheduling operations, serial communication, and menu handling.

The AVR architecture was selected because of:

- Low hardware cost
- Integrated ADC subsystem
- USART communication support
- Sufficient GPIO resources
- Ease of embedded development

The controller operates using an external crystal oscillator configured at 8 MHz.

4.2.2 Real-Time Clock Module

A DS1307 Real-Time Clock module was integrated into the system to provide continuous time and date information.

The RTC module communicates with the microcontroller using the I²C protocol and maintains timekeeping independently of the main processor execution cycle.

The RTC subsystem provides:

- Hour, minute, and second tracking
- Date and calendar information
- Time-based relay scheduling
- Autonomous time retention

4.2.3 LCD Interface

A character-based alphanumeric LCD module was used as the primary user interface.

The LCD displays:

- Current date and time
- Temperature readings
- Menu options
- Security warnings
- Device status information

The display was interfaced using parallel GPIO connections.

4.2.4 Keypad Interface

Multiple push buttons were connected to the GPIO pins of the AVR microcontroller to provide menu navigation and parameter configuration capabilities.

The keypad subsystem allows the user to:

- Configure scheduling parameters
- Adjust temperature settings
- Enter passwords
- Navigate system menus
- Activate protection modes

4.2.5 Temperature Sensor

An analog temperature sensor was connected to one of the AVR ADC channels.

The measured analog voltage is converted into digital values through the internal ADC subsystem and processed by the automation software.

The temperature sensor is used for:

- Environmental monitoring
- Automatic cooling activation
- LCD temperature display

4.2.6 Photocell Sensors

Photocell sensors were integrated into the system for environmental light detection.

The sensors generate analog voltage levels proportional to surrounding light intensity. These values are sampled through the ADC subsystem and used for automation logic related to lighting conditions.

Applications include:

- Indoor illumination monitoring
- Outdoor darkness detection
- Lighting automation

4.2.7 Laser Protection Sensor

A laser-based intrusion detection mechanism was implemented as part of the security subsystem.

The protection system consists of:

- Laser transmitter
- Optical receiver
- Alarm output

Interruption of the laser path generates a security event which activates the buzzer alarm when protection mode is enabled.

4.3 Receiver Board Hardware

The receiver subsystem was designed as a lightweight actuator controller responsible for external device switching.

The receiver board receives serial packets from the transmitter unit and controls output peripherals through GPIO pins.

The receiver subsystem includes:

- AVR microcontroller
- USART communication interface
- Relay driver circuits
- LCD status display
- Output switching ports

4.3.1 Relay Driver Circuit

External electrical devices were controlled using relay switching circuits connected to the AVR output pins.

The relay subsystem controls:

- Lighting systems
- Cooling devices
- Alarm outputs
- Auxiliary electrical loads

Because the AVR GPIO pins cannot directly drive relay coils, transistor-based driver stages were used between the microcontroller and relay modules.

4.3.2 Buzzer Alarm

An electronic buzzer was integrated into the security subsystem to provide audible alarm indication during intrusion events.

The buzzer is activated when:

- Protection mode is enabled
- Laser interruption is detected
- Security violations occur

4.4 Power Supply Design

The system was powered using a regulated DC power supply providing stable operating voltage for all embedded components.

Voltage regulation circuitry was used to ensure reliable operation of:

- AVR microcontrollers
- RTC module
- LCD display

- Sensors
- Relay circuits

Special consideration was required for relay switching circuits in order to minimize electrical noise and voltage fluctuations.

4.5 Communication Interface

Communication between the transmitter and receiver boards was implemented using asynchronous USART serial communication.

The communication interface provides:

- Reliable short-distance communication
- Simple hardware implementation
- Low computational overhead
- Real-time command transmission

The serial interface operates at 9600 baud using standard TTL voltage levels.

4.6 Hardware Design Considerations

Several engineering considerations influenced the hardware implementation:

- Low system cost
- Component availability
- Simplicity of implementation
- Expandability
- Embedded resource limitations
- Reliability under continuous operation

The final implementation successfully integrated sensing, communication, automation, and security functionalities into a standalone embedded hardware platform using low-cost electronic components.

Chapter 5

Experimental Results and System Evaluation

The developed smart home automation and security system was experimentally evaluated under multiple operating conditions in order to verify the reliability of sensing, communication, automation, and security functionalities.

The implemented prototype successfully demonstrated real-time monitoring, scheduled automation, remote device control, and security event handling using AVR-based embedded hardware.

5.1 Experimental Setup

The experimental platform consisted of two AVR-based hardware boards connected through USART serial communication. The transmitter board was connected to multiple sensors and user interface modules, while the receiver board controlled relay-driven output devices.

The experimental setup included:

- AVR transmitter/controller board
- AVR receiver/actuator board
- DS1307 RTC module
- Temperature sensor
- Photocell sensors
- Laser security sensor
- Relay modules
- LCD display
- Keypad interface
- Buzzer alarm

The system was tested under continuous operation to evaluate communication reliability and automation performance.

5.2 Relay Scheduling Evaluation

The relay scheduling subsystem was tested using multiple programmable ON/OFF intervals.

The RTC module continuously provided current time information, while the software compared the active time against user-defined schedules.

The experiments demonstrated that:

- Relay activation occurred at the programmed start time
- Relay deactivation occurred at the programmed stop time
- Multiple scheduling configurations operated correctly
- Long-duration operation remained stable

The scheduling mechanism successfully enabled autonomous device control without external intervention.

5.3 Temperature Control Performance

The temperature monitoring subsystem was evaluated using varying analog input levels applied to the ADC channel.

The system continuously displayed measured temperature values on the LCD interface and compared them against a configurable threshold value.

Experimental observations confirmed that:

- Temperature readings were updated correctly
- ADC conversion operated reliably
- Cooling control commands were generated automatically
- The cooling subsystem activated when the measured temperature exceeded the threshold value

The temperature automation mechanism operated successfully under repeated testing conditions.

5.4 Light Detection Evaluation

Photocell sensors were tested under different illumination conditions in order to evaluate environmental light monitoring capabilities.

The system correctly distinguished between:

- Bright environmental conditions
- Low-light conditions
- Indoor illumination changes
- Outdoor darkness conditions

The measured sensor values were processed successfully by the automation subsystem and could be used for lighting-related control decisions.

5.5 Security System Evaluation

The laser-based intrusion detection subsystem was tested by intentionally interrupting the optical signal path while protection mode was enabled.

The experiments confirmed that:

- Laser interruption was detected correctly
- Alarm conditions were generated immediately
- The buzzer activated successfully
- Password verification was required to disable protection mode

The protection subsystem demonstrated reliable operation for basic intrusion detection scenarios.

5.6 Communication Reliability

USART communication between the transmitter and receiver boards was tested under continuous data transmission conditions.

The communication protocol successfully achieved:

- Stable serial communication
- Correct packet synchronization
- Reliable command transmission
- Successful checksum validation
- Real-time actuator control

The implemented checksum mechanism successfully detected basic transmission errors during experimental testing.

5.7 User Interface Evaluation

The LCD and keypad interface were evaluated through repeated user interaction tests.

The interface successfully supported:

- Time and date configuration
- Relay scheduling configuration
- Password entry
- Temperature adjustment
- Security mode activation

The menu-driven architecture provided acceptable usability within the limitations of an embedded LCD interface.

5.8 System Stability

The system was operated continuously for extended periods in order to evaluate overall stability and reliability.

Experimental operation demonstrated that:

- The RTC subsystem maintained stable timing
- Sensor acquisition remained functional
- Communication errors were infrequent
- Relay control remained reliable
- LCD updates operated correctly

No critical software failures were observed during normal operating conditions.

5.9 System Limitations

Although the implemented prototype successfully achieved the primary project objectives, several technical limitations were identified during evaluation:

- Polling-based execution reduced processing efficiency
- The checksum algorithm provided limited error detection capability
- The system lacked wireless communication support
- Password information was stored only in volatile memory
- The software structure was not modularized extensively

These limitations are primarily related to hardware constraints and the embedded software methodologies available during the original development period.

5.10 Possible Improvements

Several improvements could be applied to modernize the system architecture and improve performance:

- Interrupt-driven event processing
- CRC-based communication validation
- Wireless communication modules
- EEPROM-based persistent configuration storage
- RTOS-based task scheduling
- Graphical user interfaces
- IoT and network connectivity
- Mobile application integration

Modern embedded platforms could significantly enhance scalability, reliability, and system flexibility.

5.11 Summary

The experimental results demonstrate that the implemented AVR-based smart home automation and security system successfully achieved the intended functionality using low-cost embedded hardware.

The system integrated real-time monitoring, relay scheduling, environmental sensing, security management, and serial communication into a standalone embedded platform. Despite the hardware and software limitations associated with early-generation embedded systems, the prototype demonstrated reliable operation and practical automation capabilities.